

# Agents as a New system Workload

Ruijie Gao, Xinyi Xu, Chenglin Li

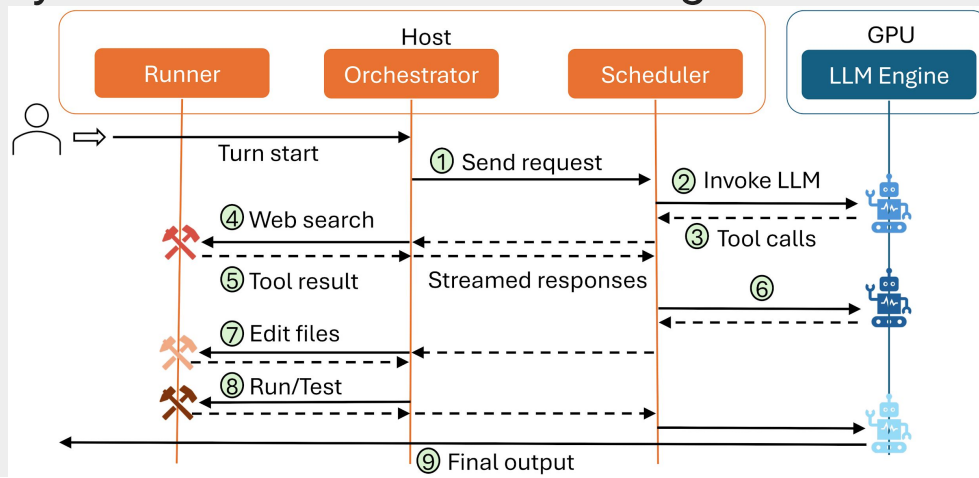
# Towards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective

**Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang,  
Tushar Krishna**

# Background and Motivation

## Why Agents Change the Systems Workload

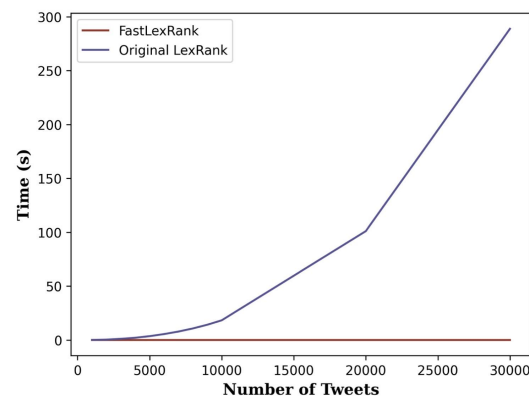
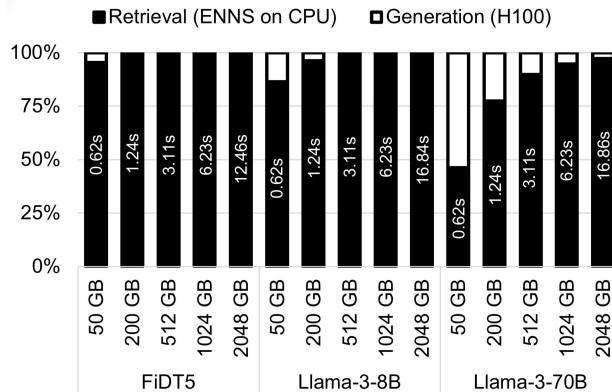
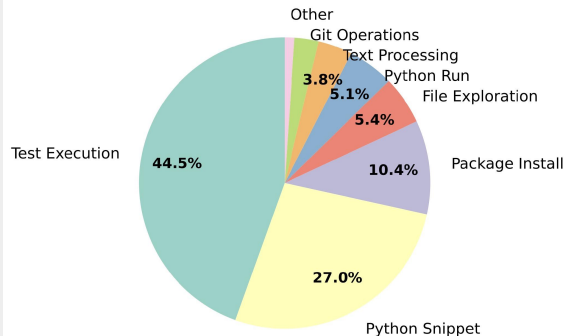
- An agent request is more than one LLM inference.
- LLM inference typically runs on the GPU.
- Tool execution and orchestration often run on the CPU.
- Agents repeatedly alternate between reasoning and tool use.



# Agent Tools Create Diverse CPU Workloads

- Representative CPU-side tools:
  - Bash / Python execution → compute + I/O intensive
  - Exact Nearest Neighbor Search (ENNS) → memory + I/O intensive
  - Lexical summarization → compute intensive
- Different tools stress different CPU resources.

(b) Bash Command Time by Category





# Agent Serving Requires Joint CPU–GPU Optimization

- GPU side is well optimized
  - PagedAttention
  - Continuous batching
  - KV-cache management
  - vLLM / SGLang
- **CPU/tool optimization also exists**
  - IKS: accelerates exact retrieval for RAG
  - Conveyor: overlaps tool execution with LLM decoding

**The missing piece: characterize where the bottleneck is,  
then jointly optimize CPU and GPU utilization.**

# Methodology

# Compile-time Characterization

| Agentic Workload                           | Compile-time Characterization |         |             | Tools                                             | Application                            |
|--------------------------------------------|-------------------------------|---------|-------------|---------------------------------------------------|----------------------------------------|
|                                            | Orchestrator                  | Path    | Flow        |                                                   |                                        |
| <b>Toolformer [59]</b>                     | LLM                           | Dynamic | Single-step | <u>Calculator API</u> , <u>Calendar</u>           | <u>MLQA</u> , <u>Math</u>              |
| <b>SWE-Agent [75]</b>                      | LLM                           | Static  | Multi-step  | <u>Bash (File I/O and Python Execution)</u>       | <u>SDE</u> , <u>Data analysis</u>      |
| <b>RAG (Haystack [14])</b>                 | Python code (Host)            | Static  | Single-step | <u>Web search</u> , <u>Document Retrieval</u>     | <u>RAG QA</u>                          |
| <b>ChemCrow [10]</b>                       | LLM                           | Dynamic | Multi-step  | <u>Conformer Gen tool</u> , <u>Reaction Tools</u> | <u>Chemistry Research</u>              |
| <b>Web-Augmented Agent (LangChain[43])</b> | Python code (Host)            | Static  | Single-step | <u>Web search</u> , <u>summarizer</u>             | <u>Web-based QA</u> ,<br><u>DevOps</u> |

- **Orchestrator:** who drives the workflow, the LLM or host Python code
- **Path:** fixed in advance (static) or decided at run time (dynamic)
- **Flow:** one pass (single-step) or repeated act–observe loops (multi-step)
- **Models:** small LMs keep the loops fast; tools handle computation and retrieval



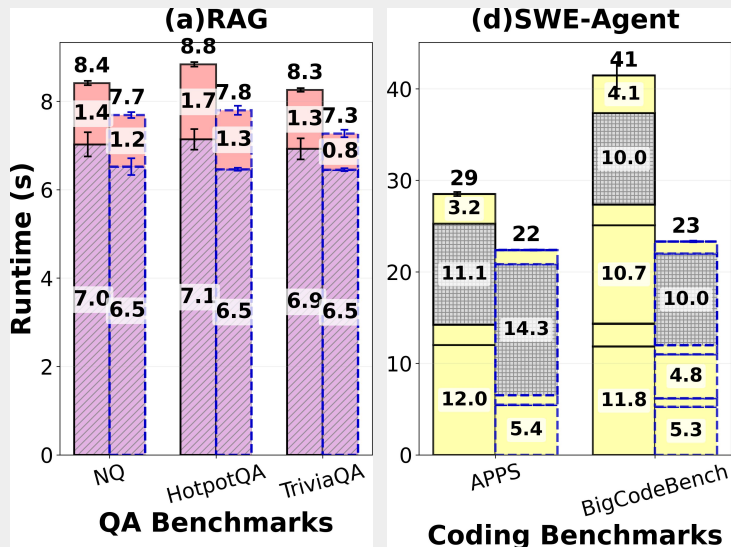
# Profiling

## Three questions, three takeaways

|                                         |                                               |                                               |
|-----------------------------------------|-----------------------------------------------|-----------------------------------------------|
| <b>Where does the time go?</b>          | CPU tools can take most of the latency        | <b>Speeding up only the GPU is not enough</b> |
| <b>How do the CPU and GPU interact?</b> | A faster GPU shifts the bottleneck to the CPU | <b>Optimize the CPU and GPU together</b>      |
| <b>How does throughput scale?</b>       | CPU tools hit their limit before the GPU      | <b>Leads to COMB and MAS</b>                  |

# Profiling

## Takeaways 1–2: CPU tools can dominate latency



### LLM Inference (GPU)

- GPT-OSS-20B (vLLM)
- Qwen2.5-Coder-32B (vLLM)

### Tools (CPU)

- ENNS Retrieval
- Bash/Python Execution

### Hardware

- System 1
- System 2

**(a) RAG:** retrieval on the CPU takes over 80% of the time

**(d) SWE-Agent:** tools reach 65% on Sys 2 (faster GPU)

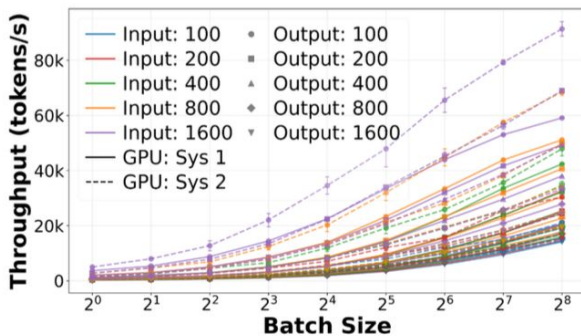
→ **Speeding up only the GPU is not enough**

Sys 1: 64-core Intel CPU + RTX Pro 6000 GPU

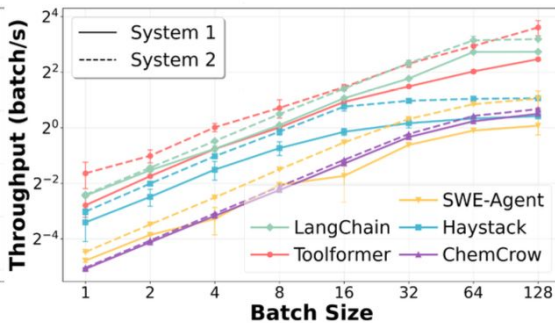
Sys 2: 72-core Grace CPU + H200 GPU (faster GPU)

# Profiling

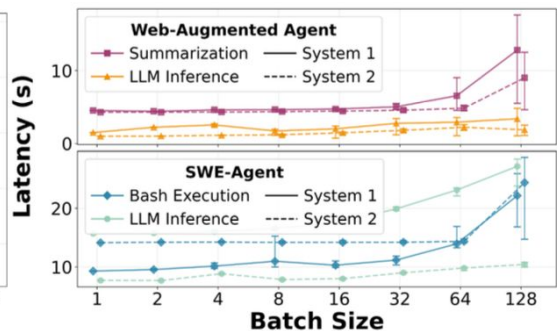
Takeaway 3: CPU tools hit their limit first → COMB and MAS



GPU only: LLM inference (tokens/s)



Full agent workflow: requests/s and stage latency



# Proposed Optimizations

## COMB (CPU-Aware Overlapped Micro-Batching): problem → idea

From profiling: CPU tool work and GPU inference run in separate phases

### The CPU gets overloaded

Too many requests at once: CPU tools compete and run  $\sim 2\times$  slower (batch 128)



### Cap the batch size

Split a large batch into smaller groups, so the CPU is not overloaded

### The CPU and GPU take turns

While tools run on the CPU, the GPU waits, and the other way around



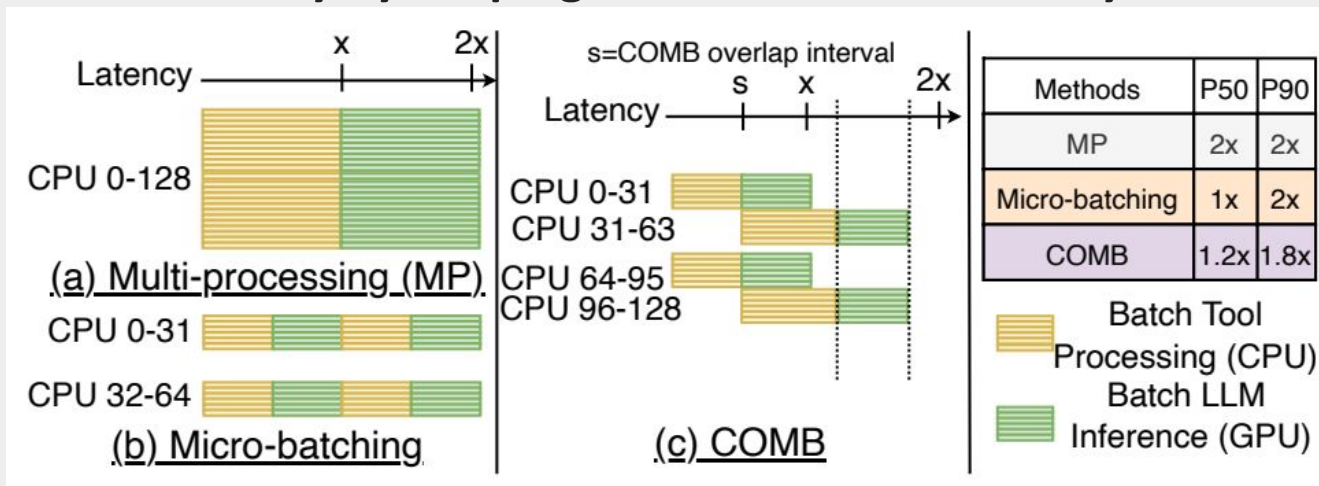
### Overlap the groups

While one group uses the GPU, the next group runs its tools on the CPU

In live serving: COMB limits how many CPU-heavy requests are active ( $N_{cap}$ )

# Proposed Optimizations

COMB cuts latency by keeping both CPU and GPU busy



Times are illustrative (paper's Fig. 4)



# Proposed Optimizations

## MAS (Mixed Agentic Scheduling): problem → idea

**Two request types:** CPU-heavy (call tools) and GPU-heavy (LLM only)

### One queue is unfair

With a single queue, the more common type takes most slots; the other waits



### A separate limit per type

Each type gets its own slots, so both the CPU and GPU stay busy

### Demand changes over time

When one type surges, its own slots alone are not enough



### A small shared reserve

Extra requests of either type can borrow from a shared pool

**Link to COMB:** the CPU-heavy limit is COMB's  $N_{cap}$

# Proposed Optimizations

**MAS: each type keeps its own slots, so neither is crowded out**

**224 active slots in total**



Slot numbers are the paper's setting

# Evaluation

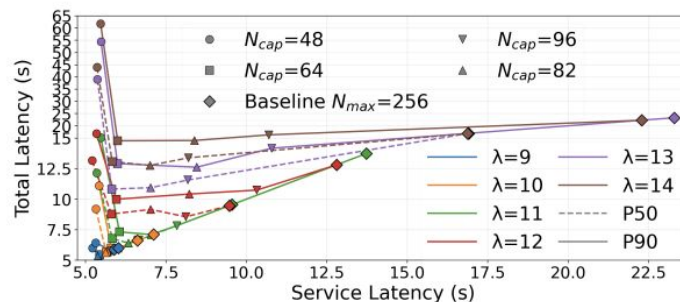
# Evaluation

## Setup: four experiments

|                            |                                                      |
|----------------------------|------------------------------------------------------|
| <b>COMB: one batch</b>     | 128 requests at once: all vs. $2 \times 64$ vs. COMB |
| <b>COMB: live serving</b>  | Requests keep arriving (Sys 2): limit 256 vs. 48–96  |
| <b>MAS: mixed requests</b> | FCFS vs. MAS with 224 slots; 25/50/75% GPU-heavy     |
| <b>Extra studies</b>       | 16-core CPU; CPU vs. GPU energy (Sys 2)              |

**Main metrics:** P50 (median) and P90 (tail) latency

## COMB Improves Latency and Throughput Together

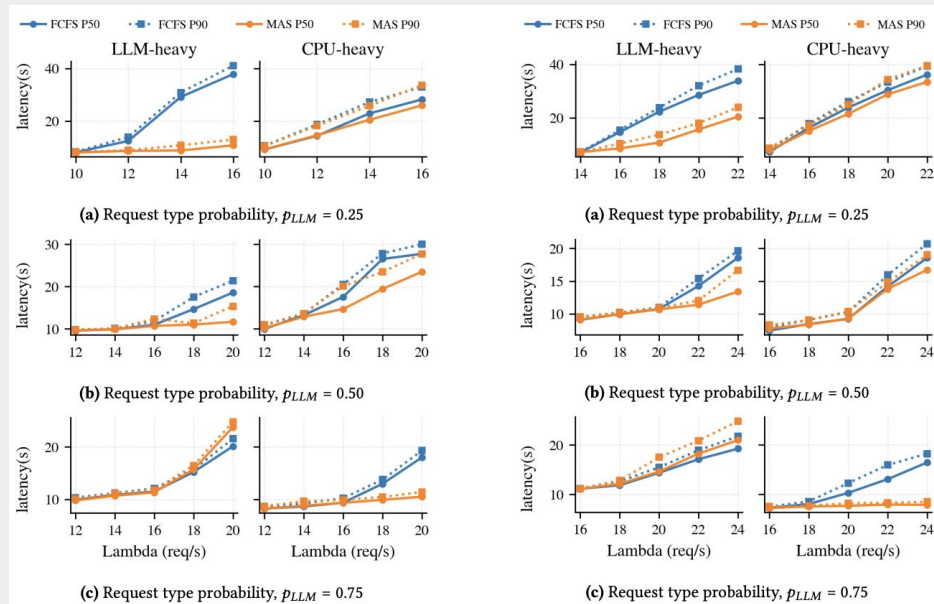


**Figure 6.** In open-loop serving system with  $\lambda = 9 - 15$  req/s arrival-rate for Web-Augmented Agent on Sys 2.

- Total latency improves by 1.6x/1.8x, and throughput improves by 1.7x
- N<sub>cap</sub>=64 keeps CPU utilization balanced under high load
- At  $\lambda=13$  req/s, COMB reduces service latency by 2.9x/3.9x at P50/P90

# MAS Protects Minority Workloads in Mixed Traffic

- With one FCFS queue, the majority request type can consume admission slots even if the minority stresses a different resource
- MAS gives CPU-heavy and GPU-heavy requests separate capacity plus a small shared reserve
- It improves minority GPU-heavy latency by up to  $2.37\times$  /  $2.49\times$ , and minority CPU-heavy latency by up to  $2.09\times$  /  $2.15\times$



Paper Fig. 7/8: MAS vs. FCFS under bursty mixed arrivals on Sys 1 and Sys 2

# Discussions



## Main Takeaways

- CPU tools can limit end-to-end performance even when GPU inference becomes faster
- COMB improves CPU-heavy serving by capping concurrency and overlapping CPU/GPU stages
- MAS protects minority request types through separate admission limits

## Limitations

- Overlap gains depend on CPU capacity and tuned caps. On the 16-core platform, micro-batching alone can be better
- Scheduling tests cover selected workloads and systems. Web-Agent uses cached HTML, excluding live network variability
- Energy profiling identifies CPU costs, but does not establish COMB/MAS energy savings

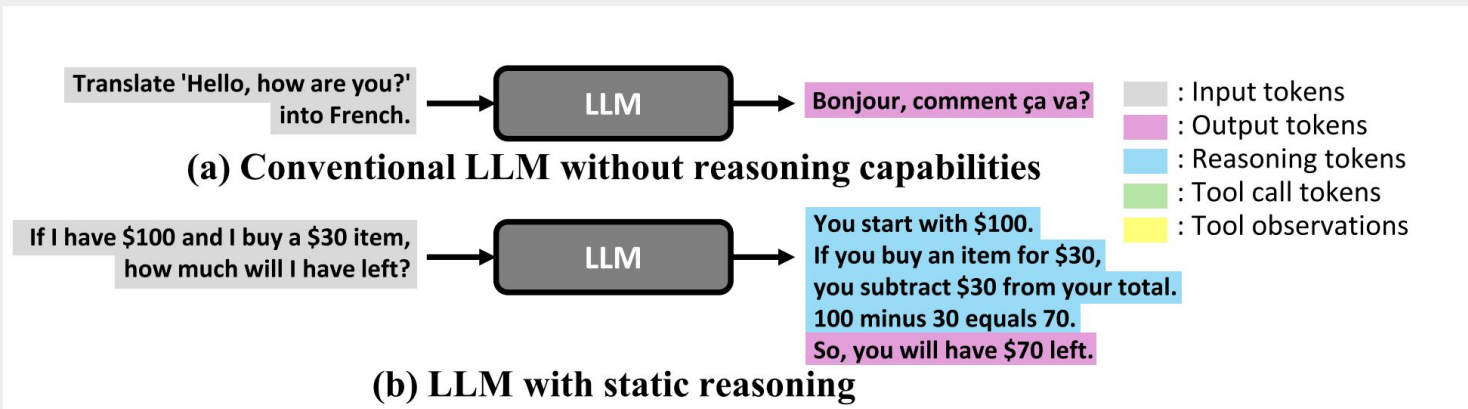
# The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective

**Jiin Kim, Byeongjun Shin, Jinha Chung, Minsoo Rhu**

# Background and Methodology

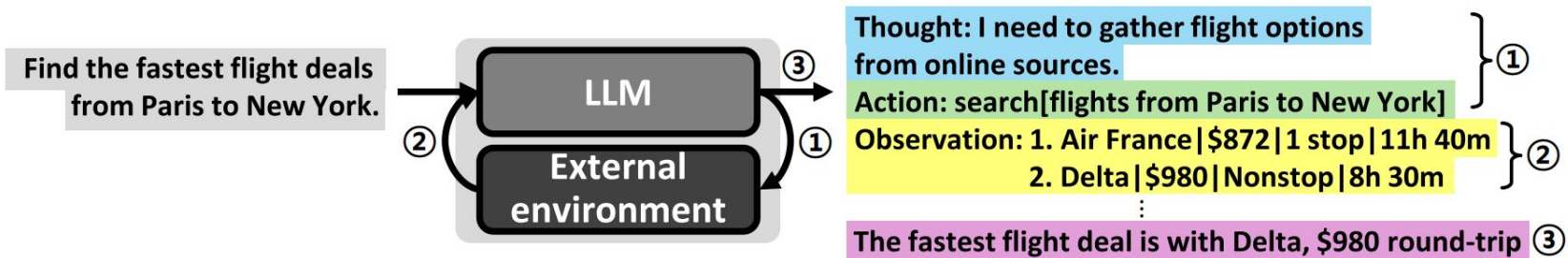
## From Test-Time Scaling to Dynamic Reasoning

- Test-time scaling improves model performance by spending more compute during inference.
- Examples: Chain-of-Thought (CoT) and Tree-of-Thought (ToT).
- AI agents extend this idea with iterative reasoning and external actions.



# AI Agents Turn Reasoning into a Dynamic Workflow

- Agents iteratively:
  - Reason → Act / Call Tool → Observe → Adapt
- Intermediate outcomes determine what happens next.
- The execution path evolves dynamically at runtime.



(c) AI agents with dynamic reasoning

## Dynamic Reasoning Changes the Serving Workload

**Dynamic control flow**  
**Execution path changes**  
**at runtime**

**Repeated LLM calls**  
**More inference within one**  
**request**

**Tool interactions**  
**External actions join the**  
**critical path**

These behaviors create variable compute, memory pressure, and latency → motivating a system-level characterization.

# Methodology

# Methodology

## Four questions, from one request to a datacenter

### One request

How much compute, time, and memory does one agent request use?



### Many requests

Can a server run many agent requests at once, efficiently?



### More reasoning

Does spending more compute give better answers?



### Datacenter

What do agents mean for energy and power at scale?

**Approach:** a measurement study. Run five representative agents on four real tasks, on a standard LLM server (vLLM), and measure the cost at each level



# Methodology

## Five representative agents

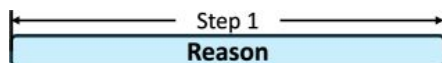
| Agent       | Reasoning | Tool use | Reflection | Tree search | Planning |
|-------------|-----------|----------|------------|-------------|----------|
| CoT         | ✓         | —        | —          | —           | —        |
| ReAct       | ✓         | ✓        | —          | —           | —        |
| Reflexion   | ✓         | ✓        | ✓          | —           | —        |
| LATS        | ✓         | ✓        | ✓          | ✓           | —        |
| LLMCompiler | ✓         | ✓        | ✓          | —           | ✓        |

**Why these five:** together they cover the main agent designs

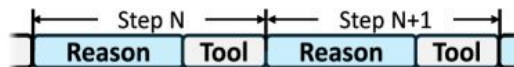
# Methodology

## How each agent runs

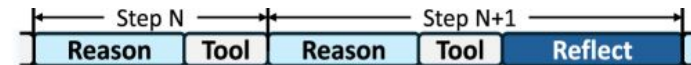
CoT



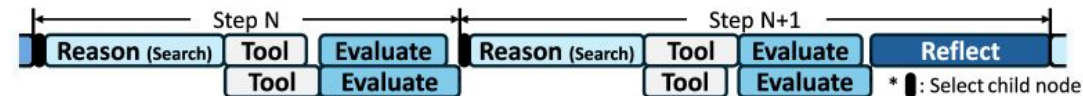
ReAct



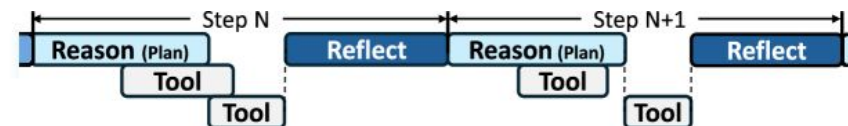
Reflexion



LATS



LLMCompiler



**Read it as:** each block is one LLM call or one tool call

# Methodology

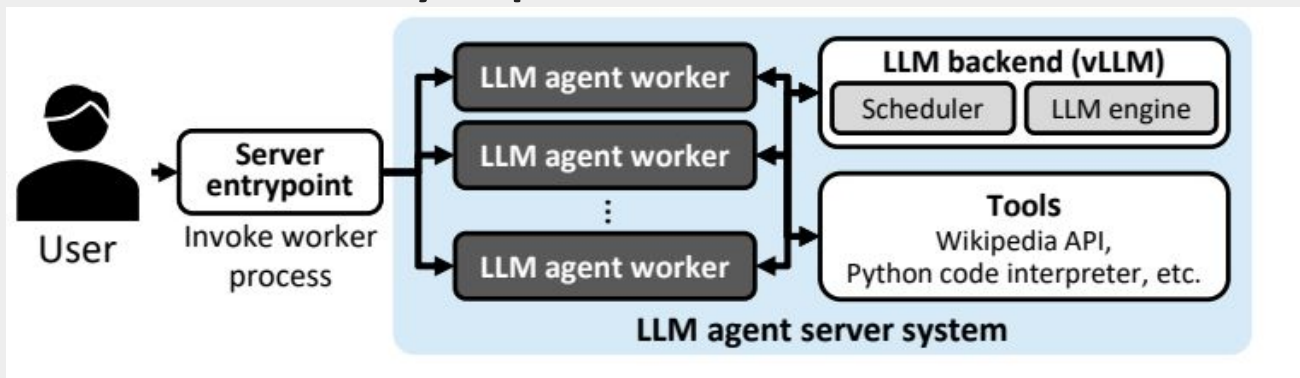
## Four tasks and a chatbot baseline

| Benchmark | Task               | Tools for the agent       | Agents              |
|-----------|--------------------|---------------------------|---------------------|
| HotpotQA  | Multi-hop Q&A      | Wikipedia API             | All five            |
| WebShop   | Online shopping    | Web search, click         | All but CoT         |
| MATH      | Math problems      | Wolfram Alpha, calculator | All but LLMCompiler |
| HumanEval | Programming        | Run own test code         | All but LLMCompiler |
| ShareGPT  | Chat conversations | None (one LLM call)       | Plain LLM           |

**Why:** each task uses different tools; ShareGPT shows a normal chatbot

# Methodology

The test server for many requests



- **Workers** run requests in parallel; each step calls the LLM or a tool
- **vLLM** batches LLM calls from all workers, first come first served



# Methodology

## Test-time scaling: letting agents reason more

### Sequential scaling

Think longer: allow more reflection steps, one after another (Reflexion, LATS)

### Parallel scaling

Think wider: explore more paths at the same time (more child nodes in LATS)

- **ReAct knobs:** step limit and number of examples in the prompt
- **Model size:** Llama-3.1 8B vs. 70B
- **For every setting:** measure accuracy and cost

# Evaluation

# Evaluation

## Models and test platform

| Resource    | 8B (default)    | 70B (size study)     |
|-------------|-----------------|----------------------|
| GPU         | 1 × A100 (40GB) | 8 × A100 (40GB each) |
| vCPUs       | 12              | 96                   |
| Host memory | 85GB            | 680GB                |

Llama 3.1 Instruct; prefix caching enabled by default.

vLLM 0.6.6 · PyTorch 2.6 · CUDA 12.8

# Evaluation

## Experiments 1–2: one request, then many

### 1. Profiling one request

What does one request cost?

- Every agent on every task, one at a time
- **Compute:** LLM/tool calls, GPU busy time
- **Time:** latency, split into LLM vs. tools
- **Memory:** tokens and KV-cache size
- **Prefix caching:** on vs. off

### 2. Serving many requests

Can a server run many requests efficiently?

- ReAct (HotpotQA, WebShop) vs. ShareGPT
- **Load:** Poisson arrivals, rising QPS
- **Compare:** one at a time vs. many at once
- **Measure:** P95 latency, peak QPS, KV cache
- **Prefix caching:** on vs. off

**Default (all experiments):** Llama-3.1 8B Instruct, prefix caching on



# Evaluation

## Experiment 3: accuracy vs. cost

| Task      | Accuracy rule         |
|-----------|-----------------------|
| HotpotQA  | Exact match           |
| WebShop   | Task score            |
| MATH      | Match, equal forms OK |
| HumanEval | Passes all unit tests |

- **50 questions** per setting
- **Cost** = end-to-end latency
- **Cost-efficiency** = accuracy  $\div$  latency
- **8B vs. 70B**: also tokens and energy

**Question:** as an agent spends more compute, how much accuracy does it gain?



# Evaluation

## Experiment 4: energy and datacenter power

- **Agents:** most accurate Reflexion and LATS on HotpotQA, with 8B and 70B; ShareGPT as the baseline
- **Measure:** GPU energy for one query (Wh)

$$\text{Power (W)} = \text{energy per query (Wh)} \times \text{queries per day} \div 24 \text{ h}$$

- **Two traffic levels:** 71.4 million queries/day (one per ChatGPT daily user) and 13.7 billion/day (Google Search scale)
- **Scope:** GPU energy only; batching across requests is not modeled

# Results

## Dynamic Reasoning Increases Per-Request Work

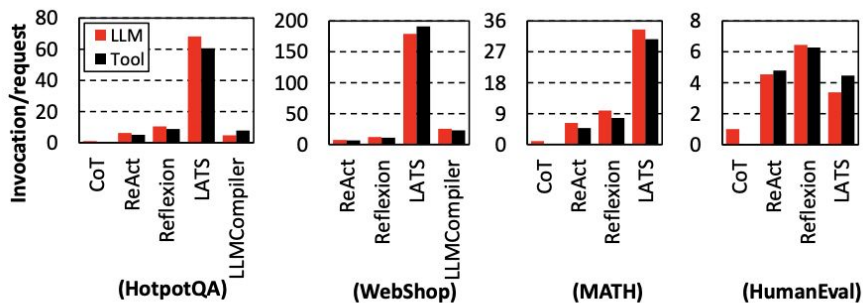


Fig. 4. Average number of LLM and tool invocations per request.

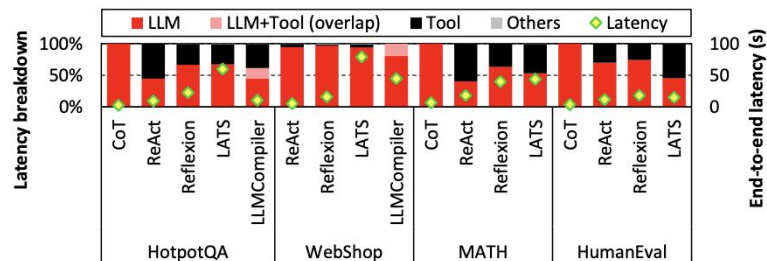


Fig. 5. Latency breakdown of agents (left axis, bar graph) and their end-to-end latency for processing a single request (right axis, diamond marker). The pink bars represent phases where LLM and tool execution latencies overlap, as observed in LLMCompiler, which asynchronously executes tools during plan generation.

- One user request can trigger many LLM and tool invocations
- Latency comes from both LLM inference and tool execution
- The sequential dependency makes a single agent request hard to parallelize internally for dependency-bound reasoning/tool stages

## Agent Serving Has Lower Throughput, but Caching Helps

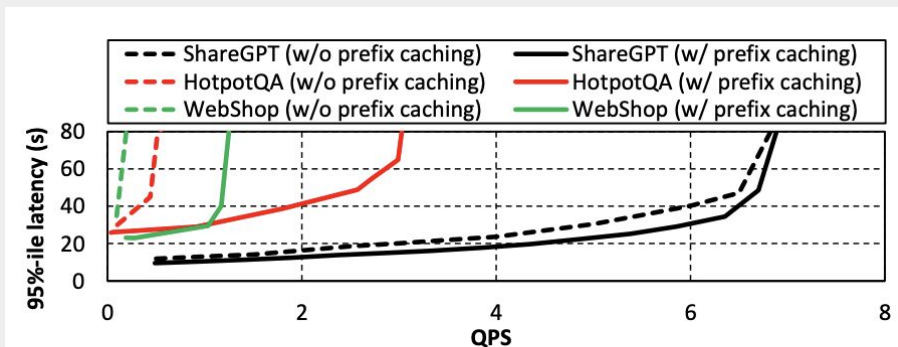


Fig. 11. 95th percentile latency for chatbot (ShareGPT) and ReAct-based AI agents (HotpotQA and WebShop) as QPS rates increase, with (solid line) and without (dashed line) prefix caching enabled.

- ReAct reaches lower sustainable QPS as each request contains repeated LLM/tool stages
- Prefix caching helps agents because many LLM calls reuse long shared prefixes
- Repeated contexts create overhead, but also make prefix caching especially valuable

## More Reasoning Shows Diminishing Returns

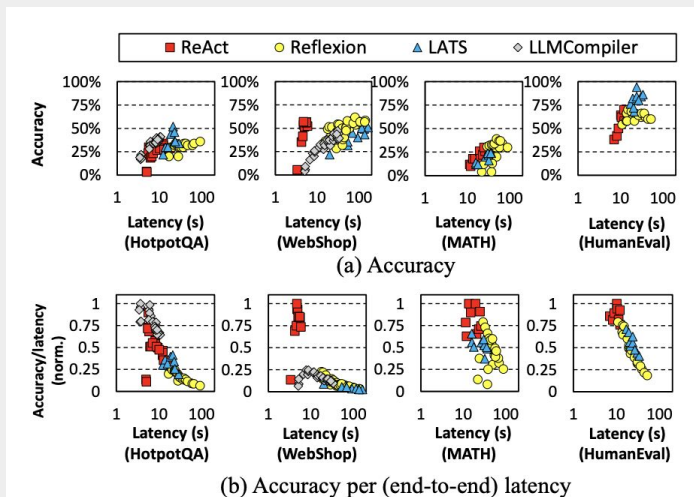


Fig. 13. Accuracy and cost-efficiency of AI agent design points. (a) Accuracy vs. latency and (b) Accuracy per latency, illustrating how efficiently each configuration translates cost into task performance.

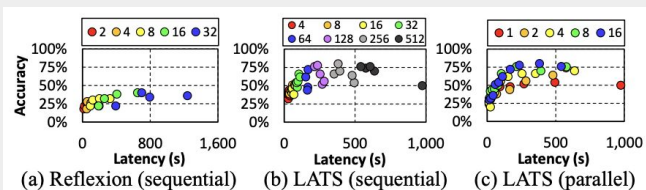


Fig. 16. Accuracy-latency trade-offs with sequential, parallel scaling on HotpotQA. Legends denote the scaling level: maximum reflection steps in (a), (b), and number of child nodes per expansion in (c).

- Higher accuracy usually requires more latency, but cost-efficiency falls after the best trade-off point
- Sequential scaling improves accuracy slowly, while parallel scaling trades memory for lower latency
- Test-time scaling should be budgeted by efficiency, not only by maximum accuracy

# Agent Test-Time Scaling Has Large Energy Cost

TABLE III

ACCURACY, LATENCY, AND GPU ENERGY CONSUMPTION WHEN SERVICING A SINGLE AGENT REQUEST ON HOTPOTQA. NUMBERS IN PARENTHESES INDICATE THE RELATIVE INCREASE OVER SHAREGPT (THE CONVENTIONAL SINGLE-TURN INFERENCE).

|     |           | Accuracy (%) | Latency (seconds) | Energy (Wh/query) |
|-----|-----------|--------------|-------------------|-------------------|
| 8B  | ShareGPT  | —            | 4.23 (1x)         | 0.32 (1x)         |
|     | Reflexion | 38           | 649.34 (153.7x)   | 41.53 (130.9x)    |
|     | LATS      | 80           | 380.90 (90.1x)    | 22.76 (71.7x)     |
| 70B | ShareGPT  | —            | 6.40 (1x)         | 2.55 (1x)         |
|     | Reflexion | 67           | 720.00 (112.6x)   | 348.41 (136.5x)   |
|     | LATS      | 82           | 305.67 (47.8x)    | 158.48 (62.1x)    |

TABLE IV

DATACENTER-WIDE POWER DEMAND UNDER CURRENT AND FUTURE TRAFFIC SCENARIOS (71.4 MILLION AND 13.7 BILLION QUERIES/DAY), ASSUMING THE HOTPOTQA BENCHMARK.

|     |           | Power @ 71.4 Million Queries/day (Watts) | Power @ 13.7 Billion Queries/day (Watts) |
|-----|-----------|------------------------------------------|------------------------------------------|
| 8B  | ShareGPT  | 1.0 M                                    | 182.7 M                                  |
|     | Reflexion | 123.6 M                                  | 23.7 G                                   |
|     | LATS      | 67.7 M                                   | 13.0 G                                   |
| 70B | ShareGPT  | 7.6 M                                    | 1.5 G                                    |
|     | Reflexion | 1.0 G                                    | 198.9 G                                  |
|     | LATS      | 471.5 M                                  | 90.5 G                                   |

- Agent-based reasoning can raise per-query GPU energy by 62.1x-136.5x
- More reasoning is a resource allocation problem, not a free path to higher quality

# Discussions



# Agents as a New System Workload

**Task quality and serving cost should be evaluated together**

## **CPU-centric execution**

CPU tools can bottleneck serving  
COMB/MAS improve CPU-GPU  
coordination

## **Dynamic reasoning**

Repeated calls raise serving cost  
Caching helps. Reasoning gains  
diminish

**Workflow design sets demand. Scheduling shapes efficiency.**